# TP Guidée : React Styles, Assets, Animations, Listes, Conditions, Formulaires, API

**Nb : TP à saisir à la main élément par élément et à bien comprendre**

## Étape 1 – Créer le fichier

1. Crée un fichier ProduitInline.js dans src/.

## Étape 2 – Code à copier-coller

```
import React from "react";

function ProduitInline(props) {
  // Ici on définit les styles en ligne avec un objet JS
  const styleProduit = {
    border: "2px solid #2196f3",
    backgroundColor: "#e3f2fd",
    borderRadius: "8px",
    padding: "10px",
    width: "220px",
    textAlign: "center",
```

```
    margin: "10px"
  };

  return (
    <div style={styleProduit}>
      <h3>{props.nom}</h3>
      <p>Prix : {props.prix} DH</p>
    </div>
  );
}

export default ProduitInline;
```

**Étape 3 – Tester le composant**

Dans App.js, ajoute :

```
import React from "react";
import ProduitInline from "./ProduitInline";

function App() {
  return (
```

```
    <div>
      <ProduitInline nom="T-shirt" prix={150} />
      <ProduitInline nom="Casquette" prix={80} />
    </div>
  );
}

export default App;
```

**Étape 4 – À tester pour mieux comprendre**

- Change la couleur de fond (backgroundColor) et la bordure.
- Modifie la largeur et observe l'effet.
- Ajoute un autre produit pour voir si les styles s'appliquent correctement.

**TP3 – Module CSS pour profil**

**But :** Éviter les conflits CSS en utilisant des modules CSS.

**Étape 1 – Créer le fichier CSS Module**

1. Crée Profil.module.css dans src/ avec le contenu :

```
.carte {
  border: 1px solid #ff9800;
  padding: 10px;
  border-radius: 6px;
  background-color: #fff3e0;
  width: 220px;
  text-align: center;
  margin: 10px;
}
.carte img {
  width: 100px;
  border-radius: 50%;
}
```

**Étape 2 – Créer le composant**

Crée ProfilModule.js :

```
import React from "react";
import styles from "./Profil.module.css";

function ProfilModule(props) {
```

```
  return (
    <div className={styles.carte}>
      <img src={props.photo} alt="Profil" />
      <h3>{props.nom} {props.prenom}</h3>
    </div>
  );
}

export default ProfilModule;
```

**Étape 3 – Tester le composant**

Dans App.js :

```
import React from "react";
import ProfilModule from "./ProfilModule";
import photo from "./photo.jpg";

function App() {
  return (
    <div>
      <ProfilModule nom="Khalid" prenom="Ali" photo={photo} />
```

```
      <ProfilModule nom="Sara" prenom="Meriem" photo={photo} />
    </div>
  );
}

export default App;
```

**Étape 4 – À tester**

- Change la couleur de fond ou la bordure dans Profil.module.css.
- Ajoute d'autres composants ProfilModule et constate qu'ils ne se mélangent pas avec d'autres CSS.

**TP4 – Galerie d'images (Assets)**

**But :** Apprendre à utiliser des images locales et externes.

**Instructions**

1. Crée Galerie.js dans src/.
2. Place 3 images dans src/images/ (ex : img1.jpg, img2.jpg, img3.jpg).

```
import React from "react";
```

```
import img1 from "./images/img1.jpg";
import img2 from "./images/img2.jpg";
import img3 from "./images/img3.jpg";

function Galerie() {
  return (
    <div>
      <h2>Galerie</h2>
      <img src={img1} alt="Image 1" width="150" />
      <img src={img2} alt="Image 2" width="150" />
      <img src={img3} alt="Image 3" width="150" />
      <img src="https://via.placeholder.com/150" alt="Externe 1" />
      <img src="https://via.placeholder.com/150/ff0000" alt="Externe
2" />
    </div>
  );
}

export default Galerie;
```

**Test**

- Vérifie que les images locales et externes s'affichent.
- Change les URLs externes pour tester d'autres images.

**TP5 – Animation au survol**

**But :** Découvrir les animations CSS avec :hover et @keyframes.

**Instructions**

1. Crée Animation.css :

```
@keyframes agrandir {
  0% { transform: scale(1); }
  50% { transform: scale(1.1); }
  100% { transform: scale(1); }
}

.bouton {
  padding: 10px 20px;
  background-color: #4caf50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
```

```
}

.bouton:hover {
  animation: agrandir 0.5s;
}
```

2. Crée BoutonAnime.js :

```
import React from "react";
import "./Animation.css";

function BoutonAnime() {
  return <button className="bouton">Survol Moi !</button>;
}

export default BoutonAnime;
```

**Test**

- Passe la souris sur le bouton et vois l'animation.
- Change la couleur ou la durée de l'animation pour expérimenter.

**TP6 – Liste de tâches (map + key)**

**But :** Transformer un tableau en JSX dynamique.

**Code**

```
import React from "react";

function ListeTaches() {
  const taches = [
    { id: 1, texte: "Faire le ménage" },
    { id: 2, texte: "Acheter du lait" },
    { id: 3, texte: "Réviser React" }
  ];

  return (
    <ul>
      {taches.map(t => <li key={t.id}>{t.texte}</li>)}
    </ul>
  );
}

export default ListeTaches;
```

**Test**

- Ajoute une tâche dans le tableau et observe l'affichage automatique.

**TP7 – Filtrage de tâches**

**But :** Combiner map() et conditions pour filtrer des éléments.

```
import React from "react";

function ListeTachesFiltrees() {
  const taches = [
    { id: 1, texte: "Faire le ménage", terminee: true },
    { id: 2, texte: "Acheter du lait", terminee: false },
    { id: 3, texte: "Réviser React", terminee: false }
  ];

  return (
    <ul>
      {taches.filter(t => !t.terminee).map(t => (
        <li key={t.id}>{t.texte}</li>
      ))}
    </ul>
```

```
  );
}

export default ListeTachesFiltrees;
```

**Test**

- Modifie terminee pour tester le filtrage.

**TP8 – Message selon rôle**

**But :** Afficher un message différent selon la prop role.

```
import React from "react";

function MessageRole({ role }) {
  return (
    <div>
      {role === "admin" ? <p>Bienvenue Admin !</p> :
       role === "user" ? <p>Bienvenue Utilisateur !</p> :
       <p>Bienvenue Invité !</p>}
    </div>
  );
}
```

```
export default MessageRole;
```

**Test**

```
<MessageRole role="admin" />
<MessageRole role="user" />
<MessageRole role="guest" />
```

**TP9 – Ajouter une tâche (Formulaire contrôlé)**

```
import React, { useState } from "react";

function AjouterTache() {
  const [taches, setTaches] = useState([]);
  const [texte, setTexte] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if(texte.trim() === "") return;
    setTaches([...taches, { id: taches.length + 1, texte }]);
    setTexte("");
  };
```

```
  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input value={texte} onChange={e => setTexte(e.target.value)}
placeholder="Nouvelle tâche"/>
        <button>Ajouter</button>
      </form>
      <ul>
        {taches.map(t => <li key={t.id}>{t.texte}</li>)}
      </ul>
    </div>
  );
}


export default AjouterTache;
```

**Test**

- Tape du texte et clique sur Ajouter → ça doit apparaître dans la liste.

**TP10 – Supprimer une tâche**

**But :** Apprendre à filtrer un tableau et mettre à jour le state.

```
import React, { useState } from "react";

function SupprimerTache() {
  const [taches, setTaches] = useState([
    { id: 1, texte: "Faire le ménage" },
    { id: 2, texte: "Acheter du lait" },
    { id: 3, texte: "Réviser React" }
  ]);

  const handleSupprimer = (id) => {
    setTaches(taches.filter(t => t.id !== id));
  };

  return (
    <ul>
      {taches.map(t => (
        <li key={t.id}>
          {t.texte} <button onClick={() =>
handleSupprimer(t.id)}>Supprimer</button>
```

```
      </li>
    ))}
  </ul>
 );
}

export default SupprimerTache;
```

**Test**

- Clique sur **Supprimer** → la tâche disparaît de la liste.

**TP11 – Formulaire multi-champs**

**But :** Gérer plusieurs champs contrôlés.

```
import React, { useState } from "react";

function FormulaireMulti() {
 const [form, setForm] = useState({ nom: "", email: "", role: "" });

 const handleChange = (e) => {
  setForm({ ...form, [e.target.name]: e.target.value });
 };
```

```
  return (
    <div>
      <form>
        <input name="nom" value={form.nom}
onChange={handleChange} placeholder="Nom" /><br/>
        <input name="email" value={form.email}
onChange={handleChange} placeholder="Email" /><br/>
        <input name="role" value={form.role}
onChange={handleChange} placeholder="Rôle" /><br/>
      </form>
      <p>Nom: {form.nom}</p>
      <p>Email: {form.email}</p>
      <p>Rôle: {form.role}</p>
    </div>
  );
}

export default FormulaireMulti;
```

**Test**

- Tape dans chaque champ → voir les valeurs mises à jour en temps réel.

**TP12 – Validation de formulaire**

**But :** Vérifier les champs obligatoires.

```
import React, { useState } from "react";

function FormulaireValidation() {
  const [nom, setNom] = useState("");
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if(!nom || !email) {
      setMessage("Nom et email obligatoires !");
    } else {
      setMessage("Formulaire envoyé !");
    }
  };
```

```
  return (
    <form onSubmit={handleSubmit}>
      <input value={nom} onChange={e => setNom(e.target.value)}
placeholder="Nom" /><br/>
      <input value={email} onChange={e => setEmail(e.target.value)}
placeholder="Email" /><br/>
      <button type="submit">Envoyer</button>
      <p>{message}</p>
    </form>
  );
}

export default FormulaireValidation;
```

**Test**

- Laisse les champs vides → message d'erreur.
- Remplis-les → message de succès.

**TP13 – Réinitialisation**

**But :** Remettre le formulaire à zéro.

```
import React, { useState } from "react";

function FormulaireReset() {
  const [nom, setNom] = useState("");
  const [email, setEmail] = useState("");

  const handleReset = () => {
    setNom("");
    setEmail("");
  };

  return (
    <form>
      <input value={nom} onChange={e => setNom(e.target.value)}
placeholder="Nom" /><br/>
      <input value={email} onChange={e => setEmail(e.target.value)}
placeholder="Email" /><br/>
      <button type="button"
onClick={handleReset}>Réinitialiser</button>
    </form>
  );
```

```
}

export default FormulaireReset;
```

**Test**

- Tape des valeurs → clique **Réinitialiser** → tout disparaît.

**TP14 – Formulaire + liste**

**But :** Ajouter des éléments à une liste dynamique.

```
import React, { useState } from "react";

function FormulaireListe() {
  const [produit, setProduit] = useState("");
  const [liste, setListe] = useState([]);

  const handleAdd = () => {
    if(!produit) return;
    setListe([...liste, { id: liste.length + 1, nom: produit }]);
    setProduit("");
  };
```

```
  return (
    <div>
      <input value={produit} onChange={e =>
setProduit(e.target.value)} placeholder="Produit" />
      <button onClick={handleAdd}>Ajouter</button>
      <ul>
        {liste.map(p => <li key={p.id}>{p.nom}</li>)}
      </ul>
    </div>
  );
}

export default FormulaireListe;
```

**Test**

- Ajoute plusieurs produits → vérifie l'affichage en liste.

**TP15 – fetch() simple**

**But :** Comprendre fetch() et récupérer des données depuis une API.

```
import React, { useState, useEffect } from "react";
```

```
function ListePosts() {
  const [posts, setPosts] = useState([]);
  const [chargement, setChargement] = useState(true);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(res => res.json())
      .then(data => {
        setPosts(data);
        setChargement(false);
      });
  }, []);

  if (chargement) return <p>Chargement...</p>;

  return (
    <ul>
      {posts.map(p => (
        <li key={p.id}>{p.title}</li>
      ))}
```

```
    </ul>
  );
}

export default ListePosts;
```

**Test**

- La liste des posts s'affiche automatiquement après quelques secondes.

**TP16 – Gestion d'erreur fetch()**

**But :** Savoir capturer et afficher les erreurs réseau.

```
import React, { useState, useEffect } from "react";

function ListePostsErreur() {
  const [posts, setPosts] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);

  useEffect(() => {
```

```
  fetch("https://jsonplaceholder.typicode.com/posts123") // URL incorrecte pour test
    .then(res => {
      if(!res.ok) throw new Error("Erreur réseau !");
      return res.json();
    })
    .then(data => {
      setPosts(data);
      setChargement(false);
    })
    .catch(err => {
      setErreur(err.message);
      setChargement(false);
    });
  }, []);

  if (chargement) return <p>Chargement...</p>;
  if (erreur) return <p>{erreur}</p>;

  return (
```

```
    <ul>
      {posts.map(p => <li key={p.id}>{p.title}</li>)}
    </ul>
  );
}

export default ListePostsErreur;
```

**Test**

- L'URL est volontairement fausse → message d'erreur s'affiche.

**TP17 – componentDidUpdate avec compteur**

**But :** Réagir aux changements du state après rendu.

```
import React, { Component } from "react";

class Compteur extends Component {
  state = { compteur: 0 };

  componentDidUpdate(prevProps, prevState) {
    if(prevState.compteur !== this.state.compteur){
      console.log("Compteur a changé :", this.state.compteur);
```

```
  }
 }

 incrementer = () => {
  this.setState({ compteur: this.state.compteur + 1 });
 };

 render() {
  return (
   <div>
    <p>Compteur : {this.state.compteur}</p>
    <button onClick={this.incrementer}>+1</button>
   </div>
  );
 }
}

export default Compteur;
```

**Test**

- Clique sur **+1** → la console affiche la nouvelle valeur à chaque clic.

**TP18 – componentWillUnmount avec timer**

**But :** Nettoyer les ressources avant suppression du composant.

```
import React, { Component } from "react";

class Timer extends Component {
  state = { secondes: 0 };

  componentDidMount() {
    this.interval = setInterval(() => {
      this.setState({ secondes: this.state.secondes + 1 });
    }, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.interval);
    console.log("Timer arrêté !");
  }
```

```
  render() {
    return <p>Temps écoulé : {this.state.secondes} s</p>;
  }
}

function AppTimer() {
  const [afficher, setAfficher] = React.useState(true);

  return (
    <div>
      {afficher && <Timer />}
      <button onClick={() => setAfficher(!afficher)}>Supprimer /
Afficher Timer</button>
    </div>
  );
}

export default AppTimer;
```

**Test**

- Clique sur le bouton → le timer s'arrête et le message s'affiche dans la console.

**TP19 – Axios GET / POST**

**But :** Maîtriser Axios et intégrer les données dans le state React.

```
import React, { Component } from "react";
import axios from "axios";

class UtilisateursAxios extends Component {
  state = { utilisateurs: [], chargement: true, erreur: null };

  componentDidMount() {
    axios.get("https://jsonplaceholder.typicode.com/users")
      .then(res => this.setState({ utilisateurs: res.data, chargement: false }))
      .catch(err => this.setState({ erreur: err.message, chargement: false }));
  }

  ajouterUtilisateur = () => {
```

```
  axios.post("https://jsonplaceholder.typicode.com/users", { name: "Nouveau", email: "nouveau@example.com" })
    .then(res => this.setState({ utilisateurs: [...this.state.utilisateurs, res.data] }));
 };

 render() {
  if(this.state.chargement) return <p>Chargement...</p>;
  if(this.state.erreur) return <p>Erreur : {this.state.erreur}</p>;

  return (
    <div>
      <button onClick={this.ajouterUtilisateur}>Ajouter Utilisateur</button>
      <ul>
        {this.state.utilisateurs.map(u => <li key={u.id}>{u.name} — {u.email}</li>)}
      </ul>
    </div>
  );
 }
```

```
}

export default UtilisateursAxios;
```

**Test**

- Clique sur **Ajouter Utilisateur** → la liste se met à jour en temps réel.

**TP20 – Compteur de clics (Classe)**

**But :** Apprendre à utiliser le state dans une **classe React** et mettre à jour l'affichage.

```
import React, { Component } from "react";

class CompteurClics extends Component {
  state = { clics: 0 };

  ajouterClic = () => {
    this.setState({ clics: this.state.clics + 1 });
  };

  reinitialiser = () => {
```

```
    this.setState({ clics: 0 });
  };

  render() {
    return (
      <div>
        <h3>Compteur de Clics</h3>
        <p>Nombre de clics : {this.state.clics}</p>
        <button onClick={this.ajouterClic}>Ajouter un clic</button>
        <button onClick={this.reinitialiser}>Réinitialiser</button>
      </div>
    );
  }
}

export default CompteurClics;
```

**Test**

- Clique sur **Ajouter un clic** → le compteur augmente.
- Clique sur **Réinitialiser** → le compteur revient à zéro.

OFPPT

**Formateur : AARAB HAFID**